Problema 1:

Dacă ne uităm la codul din iterator observam ca:

```
struct Iterator {
        int * a;
        int poz;
        Iterator(int *_a, int _poz) { a = _a; poz = _poz; }
        Iterator& operator++() { poz = (poz + 3) % 10; return *this; }
        int operator*() { return a[poz]; }
        bool operator==(const Iterator& it) { return poz == it.poz; }
};
```

- Nu exista operatorul != (necesar pentru ca un interator sa funcţioneze cu funcţiile **begin**() si **end**())

Răspuns corect **D (programul nu compilează pentru ca nu exista operatorul != in clasa Iterator)**. Nu se iau în considerare nici un alt răspuns. Se punctează răspunsul doar dacă e precedat de explicaţia corespunzătoare.

Problema 2:

Avem:

```
    auto f1 = [&](int x, int y) { a += (x + y); return x + y + a; };
    auto f2 = [=](int x, int y) mutable { a += (x - y); return (x - y) + a; };
```

Primul lambda → **f1** (captează prin referinţă (&)) deci modificarea datei membru "a" în lamba va modifica şi variabila locala "a" din funcţia main.

Al doilea lambda → **f2** (captează prin valoare (=)) şi este şi **mutable** deci se va face o copie a lui a la crearea lui f2 şi apoi apelul succesiv a lui f2 va modifica doar acea copie fără sa afecteze şi variabila locala "a" din funcţia main.

Avem:

Cod	main::a	f1::a	f2::a
int a = 10	10		10
a += f1(1, 2)	a = 13 a+=16 ⇒ 13+16 = 29	a += (x + y); ⇒ a = 13 (a din main) return x + y + a; ⇒ 1+2+13 =16	
a += f2(2, 1)	a += 12 ⇒ 29+12 = 41		a += (x - y); ⇒ a = 10 (a-ul local) + 2-1 = 11 return (x - y) + a => 2-1+11 = 12

Răspuns corect **E (41)**. Se puncteaza:

- 1 pct calculul lui f1(1,2)
- 1 pct calculul lui a rezultat din a+= f1(1,2)
- 1 pct calculul lui f2(2,1)
- 1 pct calculul lui a rezultat din a+=f2(2,1)

Nu se iau în considerare nici un alt răspuns. Se punctează răspunsul doar dacă e precedat de explicaţia corespunzătoare.

Problema 3:

Avem următorul cod în main:

```
int main() {
	A a(3);
	int res = (!a, a);
	a[(a, 'A')] += res;
	std::cout << *(reinterpret_cast<int *>(&a)) << std::endl;
	return 0;
}
```

Deci:

- a(3) ⇒ se aplica constructorul a.value = 3 x 3 = 9
- res = (!a , a) ⇒ se aplica pe rand:
  - !a ⇒ se apelează operatorul de negare (`int operator! () { return (value+=2) / 2; }`) care incrementează valoarea lui a cu 2 (deci a.value = 9+2 = 11) şi returnează jumătate din aceasta valoarea, deci 11/2 = 5 (pe int)
  - Deci avem (!a, a) ⇒ (5,a) ⇒ se aplica operatorul lista (`int operator, (int i, const A& a) { return a.value - i; }`) ⇒ (5,a) = a.value (care e 11) - 5 = 6
  - Deci res = (!a, a) ⇒ (5,a) = 6
- a[(a, 'A')] += res ⇒ se aplica pe rand:
  - (a,'A') ⇒ se apelează operatorul lista (`char operator, (char c) { return c + value % 5; }`) ⇒ (a, 'A') = 'A' + a.value (care e 11) % 5 = 'A' + 1 = 'B'
  - a['B'] ⇒ se apelează operatorul de indexare (`int& operator[] (char c) { value += (c-'A'); return value; }`) ⇒ a.value += ('B' - 'A') ⇒ a.value += 1 ⇒ a.value = 12.
  - In final a[(a, 'A')] returnează o referinţă la a.value (care este 12) peste care se adaugă 6 (valoarea lui "res") ⇒ deci a.value = 18
- Ultima parte: (`std::cout << *(reinterpret_cast<int *>(&a)) << std::endl;`) practic transforma pe a într-un pointer de tip integer, ia prima valoare de tip integer de la acea adresa şi o afişează. Cum ins clasa A conţine un singur integer (pe value) , codul din **reinterpret_cast** o sa obţină exact acea valoare. Codul afişează 18.

Răspuns corect **D (18)**. Se punctează cu 2pct calculul lui res (adică 6) şi cu încă 2 puncte calculul final.Nu se iau în considerare nici un alt răspuns. Se punctează răspunsul doar dacă e precedat de explicaţia corespunzătoare.

Problema 4:

Avem următorul cod în main:

```
delete reinterpret_cast<A*>(new C());
```

Deci - se crează un obiect de tipul C, care apoi e convertit folosind un **reinterpret_cast** la un obiect de tipul A care apoi este şters. Clasa C este derivată din clasa A şi are şi un membru de tipul B. Ceea ce înseamnă ca pentru crearea clasei C se vor apela în ordine: constructorul de la A(din cauza derivării), apoi constructorul din B (dacă membru în clasa C) şi apoi cel din C. Practic se va afişa A,B,C
Pentru ca avem de a face un un **reinterpret_cast** la A*, se va păstra exact acelaşi pointer dar tipul se schimba. Cum A nu are **vfptr** (destructorul lui A **NU** este virtual) - compilatorul nu mai ştie sa apeleze şi ceilalţi destructori deci îl va apela doar pe a lui A. Prin urmare se va afişa A,B,C,~A

Răspuns corect **E (A,B,C,~A)**. Se punctează 2 puncte pentru secventa de constructori (A,B,C) şi 2 puncte pentru destructor (~A). Se punctează răspunsul doar dacă e precedat de explicaţia corespunzătoare.

Problema 5:

Avem următorul cod în main:

```
int A::data[5] = { 0,1,2,3 };
int main() {
	A a, b;
	A* a1 = new A();
	A* a2 = new A();
	A *a3 = new A();
	A *a4 = new A();
	std::cout << (int)(*a1) << ",";
	std::cout << (int)(*a2) << ",";
	return 1;
}
```

In acest caz avem următoarele lucruri de care trebuie sa ţinem cont:

- Constructorul (`A() { value++; for (int tr = 1; tr < 4; tr++) data[tr]++; }`) care incrementează valoarea din value, şi creste cu +1 toate valorile din data[1], data[2] si data[3]
- Operatorul new (care se aplica doar pentru obiectele alocate pe heap - **NU SI PENTRU CELE ALOCATE PE STIVA** - (`void* operator new(size_t sz) { auto poz = data[0]; data[0] = (data[0] + 1) & 3; return &data[poz+1]; }`) care citeşte o poziţie din data[0], o incrementează şi dacă ajunge la valoarea 3 o face din nou 0, şi apoi returnează un pointer către data[pozitie+1]. Practic va returna fie adresa lui data[1], fie adresa lui data[2], fie adresa lui data[3]

Deci avem următoarele stagii în execuţie:

Cod	A::data[0]	A::data[1]	A::data[2]	A::data[3]	A::data[4]	a.value	b.value
	0	1	2	3	0		
A a		2	3	4	0	?	?
A b		3	4	5		?	?
`a1=new A()` `puncteaza la data[1]`	1	5	5	6	0		
`a2=new A()` `puncteaza la data[2]`	2	6	7	7	0		
`a3=new A()` `puncteaza la data[3]`	3	7	8	9	0		
`a4=new A()` `puncteaza la data[4]`	0	8	9	10	1		

La afişare avem următoarele situaţii:

- (`std::cout << (int)(*a1) << ",";`) ⇒ apelează operatorul de cast la int (`operator int () { (*((&value) + 1))++; return value; }`) ⇒ care practic ia adresa curenta (ca si pointer de tipul int*) o incrementează deci se duce la următoarea adresa după care incrementează valoarea de la acea adresa şi returnează valoarea curenta.
- In cazul lui a1 ⇒ pentru ca a1 punctează la A::Data[1] se va duce la următorul int (deci A::Data[2]) şi îl va incrementa. Practic data[2] va avea valoarea 10 după aceasta execuţie. Codul va afişa 8 (valoarea lui Data[1].

- Într-un mod similar se întâmplă şi la castarea lui (*a2) ⇒ in acest caz se va afişa valoarea lui a2 (adică Data[2] care este 10 şi se va incrementa pe Data[3] la 11

Codul afişează 8,10 pe ecran.
Răspuns corect **E** . Se punctează 1 punct calculul lui data[0..4] după iniţializarea celor doua variabile statice ("a" şi "b") 2 puncte pentru calcul lui data[0..4] pana înainte de **cout**, şi 1 punct pentru ce se afişează pe ecran. Nu se dau alte punctaje suplimentare (de exemplu dacă se calculează cum arata vectorul Data după primele 2 alocări !!!). Se punctează răspunsul doar dacă e precedat de explicaţia corespunzătoare.

Problema 6:

Aici se face o iteraţie de la 0 la 19 (inclusiv) şi dacă numărul & 3 returnează 1 se arunca o excepţie şi se afişează numărul. Mai exact, se afişează toate numerele de la 0 la 19 dacă număr & 3 = 1. Numerele care respecta aceasta caracteristica sunt 1,5,9,13,17
Răspuns corect **D (1,5,9,13,17)**. Nu se iau în considerare nici un alt răspuns. Se punctează răspunsul doar dacă e precedat de explicaţia corespunzătoare.

Problema 7:

Principiul închis-deschis (open-close):
entităţile software (module, clase, funcţii etc.) trebuie să fie deschise la extensii şi închise la modificare (Bertrand Meyer, 1988).
Partea "deschis la extensii" se referă la faptul că poate fi extinsă comportarea modulului prin adăugarea de noi funcţionalităţi pentru a satisface noi cerinţe.
Partea "închis la modificare" se referă la faptul că nu este permisa modificarea codului sursă atunci când se adaugă noi funcţionalităţi. În acest mod se evită situaţia neplăcută când adăugarea de noi funcţionalităţi poate altera comportamentul celor existente.
De cele mai multe ori, acest principiu este aplicat împreună cu cel al substituirii (Liskov).
Se punctează:

- **2 pct** pentru descrierea principiului
- **1 pct** pentru detalierea componentei "deschis"
- **1 pct** pentru detalierea componentei "închis"